Generics

Type parameters, generic classes and methods, bounds, and wildcards

---

You have used generic types since the first week of the course, every time you wrote `ArrayList<String>`. This part explains what the angle brackets do, why Java has them, and how to write generic classes and methods of your own. It ends with bounds and wildcards, which let a generic method say exactly which types it accepts, and with the few things generics cannot do. For reading, see Core Java, Volume I, Chapter 8. In class, the Speedrun Leaderboard example builds a generic class in Checkpoint 3.

A note about these notes. I wrote these notes with help from AI, from the examples and explanations I use in class. The AI drafted much of the text and code, and I review and revise them. Every example was compiled and run, and every output and error message shown is real; long stack traces are shortened. If something looks wrong, does not match what you see in Eclipse, or contradicts the book, tell me. The textbook is the tiebreaker, and the compiler beats us both. These notes do not replace the assigned chapters.

1. The Examples

The examples use one small record and a playlist of seven songs. The songs and artists are invented.

```
public record Song(String title, String artist, int seconds) { }
```

Table 1: The playlist used in the examples, in its original order.

Title	Artist	Seconds
Midnight Drive	Neon Harbor	254
Paper Moons	The Lanterns	198
Slow Burn	Neon Harbor	312
Northern Lights	Ada Vale	287
Echo Park	The Lanterns	221
Glass Garden	Ada Vale	345
Static	Neon Harbor	176

`Playlist.songs()` returns these songs as a `List<Song>` that cannot be changed.

2. Why Generics?

Before Java 5, a collection held plain `Object`s. That is still possible, and it is called a *raw type*:

```
ArrayList titles = new ArrayList();
titles.add("Paper Moons");
titles.add(42);                          // nothing stops this
for (Object o : titles) {
```

```
    String title = (String) o;            // every element needs a cast
    System.out.println(title.toUpperCase());
}
```

```
PAPER MOONS
Exception in thread "main" java.lang.ClassCastException: class java.lang.Integer cannot be cast
    to
class java.lang.String (...)
```

The list accepted a number, and the mistake was found only when the program ran, at the cast. A *type argument* tells the compiler what the list holds:

```
ArrayList<String> titles = new ArrayList<>();
titles.add("Paper Moons");
titles.add(42);
```

```
error: incompatible types: int cannot be converted to String
```

Now the same mistake is a compile error, found before the program ever runs, and no casts are needed, because `titles.get(0)` is known to be a `String`. That is the whole purpose of generics: *the compiler checks the types for you*. Raw types still compile, so that old code keeps working, but Eclipse marks every one with a warning. Do not write them.

3. Using Generic Types

In `List<String>`, `List` is a *generic type* and `String` is its type argument. A `Map` takes two, one for the keys and one for the values, as in `Map<String, Integer>`. When you create the object, the *diamond* `<>` lets the compiler fill in the type arguments from the variable's type:

```
Map<String, List<Song>> songsByArtist = new TreeMap<>();
```

Type arguments must be classes

A type argument must be a class or interface, never a primitive type such as `int`:

```
List<int> lengths;
```

```
error: unexpected type
  required: reference
  found:    int
```

Every primitive type has a *wrapper class* for this purpose, shown in Table 2.

Table 2: Primitive types and their wrapper classes.

Primitive	Wrapper class
`int`	`Integer`
`double`	`Double`
`boolean`	`Boolean`
`char`	`Character`
`long`	`Long`

Java converts between the two automatically. Putting an `int` into a `List<Integer>` *boxes* it into an `Integer`, and taking it out *unboxes* it back:

```
List<Integer> lengths = new ArrayList<>();
lengths.add(254);                  // the int is boxed into an Integer
lengths.add(198);
int total = 0;
for (int seconds : lengths) {      // each Integer is unboxed into an int
    total += seconds;
}
System.out.println("Total: " + total);
lengths.add(null);
int last = lengths.get(2);         // unboxing null
```

```
Total: 452
Exception in thread "main" java.lang.NullPointerException: Cannot invoke
"java.lang.Integer.intValue()" because the return value of "java.util.List.get(int)" is null
```

The last line shows the one trap: an `Integer` can be `null`, and an `int` cannot, so unboxing a `null` throws a `NullPointerException`.

4. Writing a Generic Class

A class becomes generic when its header declares one or more *type parameters* in angle brackets. Inside the class, each type parameter is used like any other type. This record keeps two values of any types together:

```
public record Pair<A, B>(A first, B second) { }
```

```
Pair<String, Integer> song = new Pair<>("Slow Burn", 312);
Pair<String, Boolean> setting = new Pair<>("shuffle", true);

String title = song.first();          // no cast needed
int seconds = song.second();
System.out.println(title + " lasts " + seconds + " seconds");
System.out.println(setting);
```

```
Slow Burn lasts 312 seconds
```

```
Pair[first=shuffle, second=true]
```

One class serves every combination of types, and the compiler still checks each use: `song.first()` is a `String`, and `song.second()` is an `Integer`. Type parameters are named with single capital letters by convention: `T` for a type, `E` for the elements of a collection, `K` and `V` for keys and values, or letters in order, as here.

A generic class with behavior

A record is the shortest way to write a generic class, but a generic class can be an ordinary class too, with private fields, a constructor, and methods. This one keeps the most recent items added to it, newest first, and drops the oldest when it is full:

```
public class RecentlyPlayed<T> {
    private final int capacity;
    private final List<T> items = new ArrayList<>();

    public RecentlyPlayed(int capacity) {
        if (capacity < 1) {
            throw new IllegalArgumentException("capacity must be at least 1");
        }
        this.capacity = capacity;
    }

    public void add(T item) {
        items.remove(item);              // played again: move it to the front
        items.add(0, item);
        if (items.size() > capacity) {
            items.remove(items.size() - 1);
        }
    }

    public T latest() {
        return items.get(0);
    }

    public List<T> getItems() {
        return List.copyOf(items);
    }
}
```

```
RecentlyPlayed<Song> songs = new RecentlyPlayed<>(3);
for (Song s : Playlist.songs()) {
    songs.add(s);
}
System.out.println("Latest: " + songs.latest().title());
for (Song s : songs.getItems()) {
    System.out.println("  " + s.title());
}

RecentlyPlayed<String> searches = new RecentlyPlayed<>(3);
```

```
searches.add("ada vale");
searches.add("slow burn");
searches.add("ada vale");
System.out.println(searches.getItems());
```

```
Latest: Static
  Static
  Glass Garden
  Echo Park
[ada vale, slow burn]
```

- **`T` is declared once, in the class header,** and then used for a field (`List<T>`), a parameter (`T item`), and a return type (`T latest()`). The constructor does not repeat it: it is `RecentlyPlayed(int capacity)`, not `RecentlyPlayed<T>(...)`.
- **One class serves both uses.** In a `RecentlyPlayed<Song>`, every `T` means `Song`; in a `RecentlyPlayed<String>`, every `T` means `String`. Adding a `String` to the first one is a compile error.
- **An item added again moves to the front** instead of appearing twice, because `items.remove(item)` finds it with `equals`. That is why "ada vale" appears only once.
- **`getItems` returns a copy,** so no caller can change the list and break the rule that it never holds more than `capacity` items.

5. Generic Methods

A single method can have its own type parameter, even in a class that is not generic. The type parameter goes in angle brackets just before the return type:

```
static <T> T lastOrDefault(List<T> list, T fallback) {
    if (list.isEmpty()) {
        return fallback;
    }
    return list.get(list.size() - 1);
}
...
List<String> titles = List.of("Paper Moons", "Static", "Echo Park");
List<Integer> none = List.of();
String last = lastOrDefault(titles, "nothing");
int lastLength = lastOrDefault(none, 0);
```

```
Echo Park
0
```

You never write the type argument when you call a generic method. The compiler *infers* it from the arguments: `T` is `String` in the first call and `Integer` in the second.

Generic methods do not have to be static

`lastOrDefault` is static only because it uses nothing but its parameters. Being generic and being static are separate choices, and a generic method that needs the object's fields is an ordinary instance method. This class keeps its own random number generator, and its `pick` method uses it:

```
public class Shuffler {
    private final Random random;

    public Shuffler(long seed) {
        random = new Random(seed);
    }

    public <T> T pick(List<T> list) {
        return list.get(random.nextInt(list.size()));
    }
}
...
Shuffler shuffler = new Shuffler(3300);
Song song = shuffler.pick(Playlist.songs());
String genre = shuffler.pick(List.of("rock", "jazz", "folk"));
Integer rating = shuffler.pick(List.of(3, 4, 5));
System.out.println(song.title() + ", " + genre + ", " + rating);
```

```
Slow Burn, folk, 5
```

`Shuffler` itself is not generic; only `pick` is, and each call gets its own `T`: `Song`, then `String`, then `Integer`. The *seed*, 3300, makes the random picks the same every time the program runs, so your output should match this one.

6. Bounded Type Parameters

Sometimes a generic method needs more from its type than "any type at all." This method finds the largest element of a list, so it must be able to compare elements:

```
static <T extends Comparable<? super T>> T largest(List<T> list) {
    T best = list.get(0);
    for (T item : list) {
        if (item.compareTo(best) > 0) {
            best = item;
        }
    }
    return best;
}
...
System.out.println(largest(List.of("Static", "Echo Park", "Slow Burn")));
System.out.println(largest(List.of(254, 198, 312)));
System.out.println(largest(List.of(LocalDate.of(2026, 3, 1),
        LocalDate.of(2025, 12, 24))));
```

```
Static
312
2026-03-01
```

The *bound* `T extends Comparable<...>` promises that every `T` has a `compareTo` method, which is what lets the method call `item.compareTo(best)`. In a bound, `extends` means "is or implements," for interfaces as well as classes. A list whose elements cannot be compared is rejected when the program is compiled. `Song` does not implement `Comparable`, so:

```
error: method largest in class Bounded cannot be applied to given types;
  required: List<T>
  found:    List<Song>
  reason: inference variable T has incompatible bounds
```

A bound on a class

A generic class can bound its type parameter too, and then every instance method can rely on the bound. This class remembers the largest item offered to it:

```
public class Best<T extends Comparable<? super T>> {
    private T best;

    public void offer(T item) {
        if (best == null || item.compareTo(best) > 0) {
            best = item;
        }
    }

    public T get() {
        return best;
    }
}
...
Best<Integer> longest = new Best<>();
for (Song s : Playlist.songs()) {
    longest.offer(s.seconds());
}
System.out.println(longest.get());

Best<LocalDate> latest = new Best<>();
latest.offer(LocalDate.of(2026, 3, 1));
latest.offer(LocalDate.of(2026, 10, 31));
latest.offer(LocalDate.of(2025, 12, 24));
System.out.println(latest.get());
```

```
345
2026-10-31
```

Neither method is static, and neither declares a type parameter of its own: both use the class's `T`, and `offer` can call `compareTo` because of the bound in the class header. The compiler checks the

bound when an object is created, so `new Best<Song>()` is rejected:

```
error: type argument Song is not within bounds of type-variable T
```

Why `? super T`?

The simpler bound `T extends Comparable<T>` works for `String` and `Integer`, but not for `LocalDate`. `LocalDate` gets its `compareTo` from an interface above it, `ChronoLocalDate`, so it is a `Comparable<ChronoLocalDate>`, not a `Comparable<LocalDate>`. With the simpler bound, the call fails to compile, with a message that mentions `ChronoLocalDate`. Writing `Comparable<? super T>` means "comparable with `T` or with some type above `T`", which accepts both kinds. Use it whenever a type parameter must be comparable; it is how the Java library declares its own methods, such as `Collections.max`.

7. Wildcards

Here is a rule that surprises almost everyone. `Integer` is a subclass of `Number`, but a `List<Integer>` is *not* a `List<Number>`:

```
List<Integer> lengths = new ArrayList<>();
List<Number> numbers = lengths;
```

```
error: incompatible types: List<Integer> cannot be converted to List<Number>
```

If it were allowed, `numbers.add(4.5)` would put a `Double` into a list that is supposed to hold only `Integer`s. A *wildcard*, written `?`, is how a method accepts a list of several related element types safely.

`? extends`: reading values out

```
static double total(List<? extends Number> numbers) {
    double sum = 0;
    for (Number n : numbers) {
        sum += n.doubleValue();
    }
    return sum;
}
...
System.out.println(total(List.of(254, 198, 312)));      // a List<Integer>
System.out.println(total(List.of(4.5, 3.0, 5.0)));      // a List<Double>
```

```
764.0
12.5
```

`List<? extends Number>` means "a list of some type that is a `Number`." Everything that comes out of it is a `Number`, so reading is safe. Adding is not, because the method does not know which type the list really holds:

```
error: incompatible types: int cannot be converted to CAP#1
```

`CAP#1` is the compiler's name for "the unknown type this list holds."

`? super`: putting values in

```
static void addZeros(List<? super Integer> list, int count) {
    for (int i = 0; i < count; i++) {
        list.add(0);
    }
}
```

`List<? super Integer>` means "a list of `Integer` or of some type above it." Any such list can hold an `Integer`, so adding is safe, and the method accepts a `List<Integer>`, a `List<Number>`, or a `List<Object>`. Table 3 sums up the rule.

Table 3: Choosing a wildcard.

Parameter	The method can	Use it when
`List<T>`	read and add `T`	the exact type matters
`List<? extends T>`	read values as `T`	values come *out* of the list
`List<? super T>`	add values of type `T`	values go *in* to the list

8. What Generics Cannot Do

Java checks generic types when it compiles, and then removes most of that information. This is called *type erasure*: when the program runs, a `List<String>` and a `List<Integer>` are both just `List`s. Erasure is why generics cost nothing at run time, and it explains their limits:

- **No primitive type arguments,** as Section 3 showed.
- **No `new T()`.** The running program does not know what `T` is, so it cannot create one:

  ```
  error: unexpected type
    required: class
    found:    type parameter T
  ```

- **No checking type arguments with `instanceof`.** The running program cannot tell a `List<String>` from any other list:

  ```
  error: Object cannot be safely cast to List<String>
  ```

9. Summary

Table 4: Generics in brief.

Idea	Key points
Why generics	Type mistakes become compile errors, and casts disappear. Never use raw types.
Type arguments	Classes only. Use wrapper classes for primitives, and remember that unboxing `null` throws.
Generic class	Type parameters in the class header, used like any type inside the class.
Generic method	`<T>` before the return type, static or not. The compiler infers `T` from the arguments.
Bounds	`T extends Comparable<? super T>` promises a `compareTo`, on a method or a class.
Wildcards	`List<A>` is not a `List<B>`, even when `A` extends `B`. Use `? extends` to read and `? super` to add.
Erasure	Type arguments are gone when the program runs: no `new T()`, no `instanceof List<String>`.

10. Try It

1. Write a generic method `static <T> T firstOrDefault(List<T> list, T fallback)`, and call it with a list of titles and with an empty list of lengths.
2. Make `Song` implement `Comparable<Song>`, comparing by title, and call `largest(Playlist.songs())`. Which song is largest?
3. Change `total`'s parameter from `List<? extends Number>` to `List<Number>`. Which calls stop compiling, and why?
4. Write a generic record `Ranked<T>` that holds a rank (an `int`) and an item of type `T`, and create a `Ranked<Song>` and a `Ranked<String>`.
5. Add an instance method `public <T> List<T> pickSeveral(List<T> list, int count)` to `Shuffler` that returns `count` random picks, and call it with the playlist and with a list of genres.